

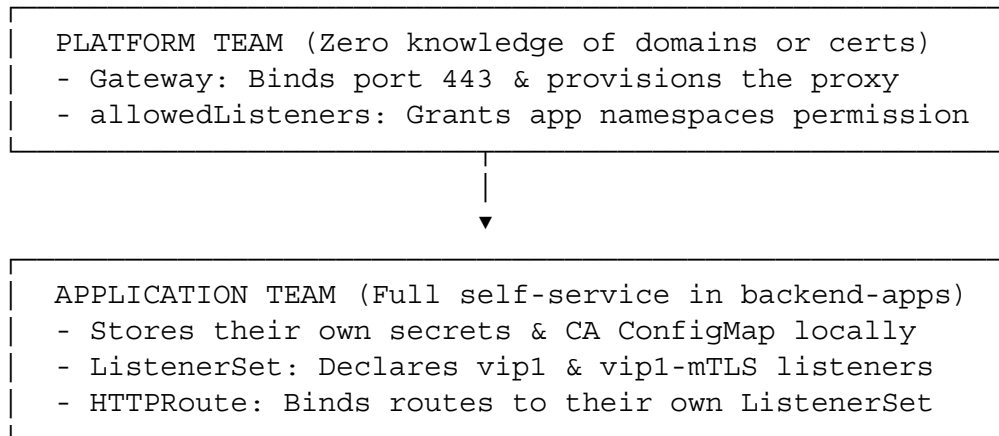
You hit the exact friction point that led to the evolution of the Gateway API.

In that previous manifest, having the Platform team define every hostname, TLS certificate, and client CA bundle inside the Gateway re-created the classic bottleneck: every new domain or cert rotation still required an Ops ticket.

Gateway API (standardized via **GEP-1713 / ListenerSet**) eliminates this dependency entirely.

The Platform team configures only the base load-balancer plumbing once, delegating both entry doors and certificate management directly to the Application team.

The Decoupled Model



1. Platform Tier: Completely Generic (Set & Forget)

The Platform team doesn't configure domains, certs, or routes. They only declare: *"Here is port 443 on the load balancer; authorized app namespaces can attach their own listeners."*

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: shared-edge-gateway
  namespace: gateway-infra
spec:
  gatewayClassName: eg-gateway-class
  # Delegation Policy: Who is authorized to define listeners?
  allowedListeners:
    namespaces:
      from: Selector
      selector:
        matchLabels:
          listener-access: "granted"
  listeners:
    # A generic base listener to bind the port on the proxy
    - name: https-base
      protocol: HTTPS
      port: 443
```

```

    tls:
      mode: Terminate
      certificateRefs:
        - group: ""
          kind: Secret
          name: fallback-default-cert # Generic fallback/sni
wildcard

```

2. Application Tier: 100% Self-Service (Namespace: backend-apps)

Everything else—the server certificates, client CA validation bundle, listener declarations, and routing—lives entirely inside the application team's repo and namespace.

A. App Team's Secrets & CA Bundle (Local to backend-apps)

```

apiVersion: v1
kind: Namespace
metadata:
  name: backend-apps
  labels:
    listener-access: "granted" # Matches Platform's allowedListeners
---
# Client CA to validate mTLS (Managed by App Team)
apiVersion: v1
kind: ConfigMap
metadata:
  name: client-ca-bundle
  namespace: backend-apps
data:
  ca.crt: |
    -----BEGIN CERTIFICATE-----
    ... (App's Client Root/Intermediate CA) ...
    -----END CERTIFICATE-----
---
# Internal cert for vip1 (Managed by App Team / cert-manager)
apiVersion: v1
kind: Secret
metadata:
  name: internal-gateway-tls
  namespace: backend-apps
type: kubernetes.io/tls
data:
  tls.crt: <BASE64_CERT>
  tls.key: <BASE64_KEY>
---
# Public cert for vip1-mTLS (Managed by App Team / cert-manager)
apiVersion: v1

```

```

kind: Secret
metadata:
  name: public-vip1-mtls-tls
  namespace: backend-apps
type: kubernetes.io/tls
data:
  tls.crt: <BASE64_CERT>
  tls.key: <BASE64_KEY>

```

B. App Team Declares Entry Doors (ListenerSet)

The application team provisions the two doors themselves, targeting the platform's root Gateway:

```

apiVersion: gateway.networking.k8s.io/v1
kind: ListenerSet
metadata:
  name: app-listeners
  namespace: backend-apps
spec:
  parentRef:
    group: gateway.networking.k8s.io
    kind: Gateway
    name: shared-edge-gateway
    namespace: gateway-infra
  listeners:
    # Door 1: Standard HTTPS
    - name: https-vip1
      protocol: HTTPS
      port: 443
      hostname: "vip1.example.com"
      tls:
        mode: Terminate
        certificateRefs:
          - kind: Secret
            name: internal-gateway-tls # Local secret!

    # Door 2: True mTLS
    - name: https-vip1-mtls
      protocol: HTTPS
      port: 443
      hostname: "vip1-mtls.example.com"
      tls:
        mode: Terminate
        certificateRefs:
          - kind: Secret
            name: public-vip1-mtls-tls # Local secret!
      frontendValidation:

```

```

caCertificateRefs:
  - kind: ConfigMap
    name: client-ca-bundle    # Local ConfigMap!

```

C. App Team Routes Traffic to the Pods (HTTPRoute)

The HTTPRoute attaches directly to the team's own ListenerSet:

```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: dual-domain-route
  namespace: backend-apps
spec:
  parentRefs:
    - group: gateway.networking.k8s.io
      kind: ListenerSet
      name: app-listeners
      # Targets both doors in their ListenerSet
  hostnames:
    - "vip1.example.com"
    - "vip1-mtls.example.com"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: core-app-svc
          port: 8080

```

What Changed?

Responsibility	Previous Pattern (Rigid Gateway)	Decoupled Pattern (ListenerSet)
New Domain / VIP	Platform ticket to update Gateway.spec.listeners.	App team edits their own ListenerSet.
TLS Certs / Rotation	Secrets in gateway-infra or complex ReferenceGrant configs.	Local Secrets inside backend-apps.
mTLS Client CA	ConfigMap placed in gateway-infra.	Local ConfigMap inside backend-apps.
Platform Scope	Bottleneck (manages hostnames, ports, certs).	Infrastructure only (IPs, Ports, Controller).